

HUNT EYE

Autonomous UAV Target Detection, Tracking & Interception

The Complete Zero-to-100 Implementation Master Plan

Explained simply • Every step • Every command • Every reason why

PROJECT	Hunt Eye — AI Autonomous Pursuit System
TYPE	100% Open Source — Nothing built from scratch
HARDWARE	RTX 3050 Laptop GPU + Windows 11
LANGUAGE	Python — all tools have Python bindings
EXPERIENCE	Beginner-friendly — explained simply throughout
PHASES	6 Phases · 15 Build Steps · ~16 weeks
GOAL	One command launches a drone that autonomously hunts a target

Version 1.0 — 2025

TABLE OF CONTENTS

CH 1 What Is Hunt Eye? The Big Picture

- › What it is
- › How it works
- › Why open source

CH 2 Every Tool Explained Simply

- › 15 tools broken down
- › Why each exists
- › Full tool table

CH 3 The Architecture

- › 7 layers
- › SharedState concept
- › Thread frequencies

PH 0 Environment Setup

- › WSL2
- › Python + CUDA
- › AirSim
- › PX4 SITL
- › Virtual env

PH 1 Project Skeleton

- › Folder structure
- › config.yaml
- › SharedState + HAL + Orchestrator
- › Camera + Dashboard

PH 2 Vision — Making the Drone See

- › YOLOv8 detector
- › MiDaS depth
- › Terrain cost map

PH 3 Tracking — Locking On

- › ByteTrack
- › TargetFilter selective lock-on

PH 4 Control — Making It Move

- › MAVLink interface
- › PID controller

PH 5 AI Brain

- › Gymnasium env
- › PPO training
- › A* path planner

PH 6 Re-ID, Safety & Real Drone

- › OSNet re-identification
- › Safety layer
- › Sim-to-real switch

APP Quick Reference

- › All commands
- › Glossary

CHAPTER 1 — WHAT IS THIS PROJECT?

What Is Hunt Eye?

Imagine you are playing a video game where a flying robot — a drone — lives inside a 3D world. It has a camera. Your software must: (1) look at the camera and find one specific target, (2) lock its eyes on that target as it moves, and (3) fly the drone to chase and intercept it — automatically, no human pressing buttons. That is Hunt Eye.

Simple Analogy

Think of a dog playing fetch. You throw the ball (the target). The dog (the drone) sees it, locks its eyes on it, and runs toward it no matter where it goes — even if it rolls behind a tree, the dog remembers it and finds it again. Hunt Eye is the dog's brain, written in code.

The Three Core Jobs

- **SEE** — Look at every camera frame and find the target using AI object detection
- **LOCK** — Keep track of that one target even when others are around
- **FLY** — Move the drone intelligently to stay on the target and close the distance

Why Simulation First?

We do not plug into a real drone on Day 1. If the code has a bug and tells the drone to fly into a wall — in simulation you just restart. With a real drone it crashes and could hurt someone. We build and test in a fake 3D world first. Switching to a real drone later is one line in a config file.

WHY: Simulation = free, safe, repeatable testing. Every professional drone company develops in simulation first.

Why Open Source?

Open source means other people already solved the hard parts and gave the code away for free. Training an AI to detect objects took Google years and millions of dollars — they published it free. Your job is to be the engineer who connects all these expert tools into one working system. That is a harder and more valuable skill than writing any single tool from scratch.

CHAPTER 2 — EVERY TOOL EXPLAINED SIMPLY

01. AirSim — "The 3D World"

What it is: This is your fake world — like a very realistic video game. It simulates physics, gravity, wind, and a 3D environment where a virtual drone flies. It gives Python a virtual camera, GPS, and flight controls. Think of it as the Matrix — a fake reality your drone lives in.

Why we need it: Microsoft built it to train drone and car AIs safely. Physics-accurate + free.

Install: `Download binary (github.com/microsoft/AirSim/releases)`

02. PX4 SITL — "The Flight Controller Brain"

What it is: PX4 is the operating system inside every serious drone. SITL = Software In The Loop — it runs the real PX4 firmware but on your computer instead of a chip. It handles the physics of flying: keeping the drone stable, handling motor speeds.

Why we need it: Industry standard used by DJI, Parrot, and military drones. Mandatory for realism.

Install: `git clone PX4-Autopilot inside WSL2`

03. YOLOv8 — "The Eyes / Object Detector"

What it is: YOLO = You Only Look Once. An AI that looks at a camera frame and in milliseconds draws boxes around every object — person, car, bicycle. Trained on millions of images. Think of it as giving the drone superhuman eyes that never blink.

Why we need it: Runs 30+ fps on your RTX 3050. Best accuracy-to-speed ratio available for free.

Install: `pip install ultralytics`

04. OpenCV — "The Image Processing Lab"

What it is: A library of 2000+ image processing functions. Resize images, draw coloured boxes, detect edges, measure motion between frames. Think of it as Photoshop for code — instant, automatic, real-time.

Why we need it: Every computer vision project uses OpenCV. The industry standard since 2000.

Install: `pip install opencv-python`

05. MiDaS — "The Depth Guesser"

What it is: Your camera gives a flat 2D image. MiDaS is an AI that looks at a 2D image and estimates how far away each pixel is. Think of it as the part of your brain that makes one eye work like two — it guesses depth from shadows, size, and blur.

Why we need it: Works from a single camera — no expensive stereo camera needed. Lightweight.

Install: `pip install timm torch`

06. ByteTrack — "The Memory / Target Tracker"

What it is: YOLOv8 detects objects in ONE frame. ByteTrack remembers which box in frame 1 is the same person in frame 2, 3, 4. It assigns each target a permanent ID number and keeps it even when the target is partially hidden.

Why we need it: Handles occlusion better than older trackers. Uses both high and low confidence detections.

Install: `git clone ByteTrack`

07. OSNet / torchreid — "The Visual Fingerprinter"

What it is: What if the target fully disappears behind a building then reappears? ByteTrack loses the ID. OSNet creates a unique 512-number visual fingerprint for the target's appearance — clothes, colours, body shape. When anyone reappears, it checks fingerprints. This is the drone's long-term memory.

Why we need it: Called Re-Identification (Re-ID). Used in security cameras worldwide.

Install: `pip install torchreid`

08. Stable-Baselines3 — "The AI Training Coach"

What it is: This library trains our AI decision-making agent using Reinforcement Learning (RL). RL means the AI learns by trial and error — tries directions, gets points when target is centred, loses points when it crashes. After millions of tries it gets very good.

Why we need it: PPO algorithm is the safest, most reliable RL starting point. Industry standard.

Install: `pip install stable-baselines3`

09. Gymnasium — "The RL Training Arena"

What it is: To train with Stable-Baselines3 we need to wrap our simulation as a standard environment. Gymnasium defines: what the AI sees (observations), what it can do (actions), and what score it gets (rewards). The rulebook for the AI's training game.

Why we need it: Created by OpenAI. Every RL project uses it.

Install: `pip install gymnasium`

10. GDAL + osmnx — "The Map Maker"

What it is: GDAL reads real terrain elevation data (how high hills are). osmnx downloads real building maps from OpenStreetMap. Together they build a cost map — a grid where each cell says how hard it is to fly through.

Why we need it: Real terrain data makes simulation physically accurate to any location on Earth.

Install: `pip install gdal osmnx`

11. pymavlink — "The Radio Transmitter"

What it is: MAVLink is the language drones speak. pymavlink lets Python send MAVLink messages — radio commands to the flight controller: 'fly north at 3 m/s', 'land now'. Works identically in simulation and on real drones.

Why we need it: MAVLink is the universal drone communication standard. Used by every PX4/ArduPilot drone.

Install: `pip install pymavlink`

12. PyQt6 — "The Live Dashboard"

What it is: PyQt6 lets you build real desktop windows with Python. Our dashboard shows: the live camera feed with tracking boxes, drone telemetry (speed, altitude, GPS), the cost map with planned path, and current system mode.

Why we need it: Used by real Ground Control Station software. Professional grade.

Install: `pip install PyQt6`

13. PyTorch — "The Neural Network Engine"

What it is: YOLOv8, MiDaS, OSNet, and the RL policy are all neural networks. PyTorch runs them. When you say 'detect objects in this image', PyTorch does millions of math operations per second using your GPU.

Why we need it: With CUDA, uses your RTX 3050 — making everything 10-50x faster than CPU.

Install: `pip install torch torchvision`

14. NumPy — "The Math Library"

What it is: Camera frames are grids of numbers. Cost maps are grids of numbers. NumPy lets Python work with huge grids incredibly fast. Almost every line of code uses NumPy somewhere. Think of it as a super-powered calculator for grids.

Why we need it: Foundation of all scientific Python code. Uses C under the hood for speed.

Install: `pip install numpy`

15. geopy — "The GPS Calculator"

What it is: For the safety layer, we calculate: how far is the drone from home? geopy does GPS distance calculations — give it two coordinates, it returns the distance in metres.

Why we need it: Essential for the geofence — the invisible radius the drone cannot cross.

Install: `pip install geopy`

Complete Tool Reference Table

TOOL	PURPOSE	HOW TO GET IT
AirSim	3D simulation world + virtual drone	Download binary — GitHub
PX4 SITL	Virtual flight controller firmware	<code>git clone PX4-Autopilot</code>
YOLOv8	Object detection — finds targets in frames	<code>pip install ultralytics</code>
OpenCV	Image processing — draws boxes, resizes	<code>pip install opencv-python</code>
MiDaS	Depth estimation from single RGB camera	<code>pip install timm torch</code>
ByteTrack	Multi-object tracking across frames	<code>git clone ByteTrack</code>
OSNet/torchreid	Re-identification fingerprinting	<code>pip install torchreid</code>
Stable-Baselines 3	RL training — PPO/SAC algorithms	<code>pip install stable-baselines3</code>
Gymnasium	RL environment wrapper / rulebook	<code>pip install gymnasium</code>
GDAL	Reads real terrain elevation files	<code>pip install gdal</code>
osmnx	Downloads building/obstacle map data	<code>pip install osmnx</code>
pymavlink	Sends flight commands to drone/sim	<code>pip install pymavlink</code>
PyQt6	Live monitoring dashboard window	<code>pip install PyQt6</code>
PyTorch	Runs all neural networks on GPU	<code>pip install torch</code>
NumPy	Array math — foundation of everything	<code>pip install numpy</code>
geopy	GPS distance for safety layer	<code>pip install geopy</code>

CHAPTER 3 — THE ARCHITECTURE

The 7-Layer Stack

The system is like a factory assembly line with 7 stations. Raw material (camera frames) enters at Layer 1. By Layer 7 it has become a flight command sent to the drone. Each layer does exactly ONE job.

LAYER 1	Simulation / HAL	The fake world OR real drone — swapped by one config line
LAYER 2	Sensor Streams	Camera frames, IMU, GPS — feeds raw data upward
LAYER 3	Perception Pipeline	YOLOv8 + OpenCV + MiDaS — sees and understands the world
LAYER 4	Target Tracking Engine	ByteTrack + TargetFilter + OSNet — locks on and remembers
LAYER 5	AI Decision Engine	PPO policy + Gymnasium — decides where to fly
LAYER 6	Path Planner	A* algorithm on cost map — routes around obstacles
LAYER 7	Flight Control + UI	MAVLink commands + PyQt6 dashboard

The SharedState — The Spine of Everything

The most important concept in the project. Every module runs in its own parallel thread. SharedState is a single Python object that every thread reads from and writes to. It is like a shared whiteboard in the middle of the factory. Nobody calls another worker directly — they just update the whiteboard.

```
# Every thread reads/writes this one object — never calls each other directly @dataclass class SharedState: lock: threading.RLock # stops two threads writing at same time uav_position: np.ndarray # [x, y, z] in metres latest_frame: np.ndarray # current camera image (numpy array) detections: list # YOLOv8 output bounding boxes active_target_id: int = -1 # which target ID we are chasing (-1 = none) target_bbox: tuple = None # (x, y, w, h) of locked target in pixels cost_map: np.ndarray = None # 2D terrain cost grid from terrain loader planned_path: list = [] # A* output waypoints list system_mode: str = "idle" # idle / tracking / returning / emergency
```

Thread Frequencies — Why Each Speed Matters

Thread	Speed	Why This Speed
camera_stream	30 Hz	Camera is 30fps — grab every frame so nothing is missed
perception_loop	15 Hz	YOLOv8 takes ~33ms on RTX 3050 — that is our realistic max
tracking_loop	15 Hz	Runs right after perception — same pace makes sense
ai_loop	10 Hz	RL policy query is fast but 10x/sec is more than enough
planner_loop	5 Hz	A* only needed when waypoint changes — 5Hz is plenty

control_loop	20 Hz	Flight controller needs frequent commands to stay stable
dashboard	10 Hz	Human eye cannot process faster than ~10fps update anyway

PHASE 0

Environment Setup — Getting Your Computer Ready

Before writing a single line of code, your computer needs the right tools installed. Think of this as building your workshop before starting construction. Do it once and never again.

STEP 0A — Install WSL2 (Windows Subsystem for Linux)

What is WSL2?

WSL2 lets your Windows computer run a full Linux OS inside a window. PX4 SITL only works on Linux. WSL2 is the bridge — Windows for your daily use, Linux for the drone tools. Think of it as a second computer living inside Windows.

WHY: PX4 does not run natively on Windows. WSL2 is the cleanest solution — no dual boot needed.

Exact Steps:

1. Open PowerShell as Administrator (right-click Start menu)

```
wsl --install
```

2. Restart your computer when prompted.
3. After restart, Ubuntu 22.04 opens. Create a username and password.
4. Verify:

```
wsl --list --verbose # You should see: Ubuntu Running 2
```

WATCH OUT: If you see version 1 instead of 2, run: `wsl --set-version Ubuntu 2`

STEP 0B — Install Python 3.10 + CUDA

What is CUDA?

CUDA is Nvidia's software that lets Python use your GPU for math. Without CUDA, YOLOv8 runs at ~5fps on CPU. With CUDA on your RTX 3050, it runs at 30+ fps. Think of CUDA as the translator between Python and your graphics card.

WHY: GPU acceleration is the difference between a barely-working system and a real-time one.

Exact Steps:

- Go to python.org → Downloads → Python 3.10.x → 64-bit installer
- Run installer — CHECK 'Add Python to PATH' before clicking Install
- Go to developer.nvidia.com/cuda-downloads → Windows → x86_64 → CUDA 11.8
- Run CUDA installer → Express installation

Verify:

```
python --version # Should print: Python 3.10.x nvcc --version # Should print: Cuda compilation tools, release 11.8
```

Install PyTorch with GPU support:

```
pip install torch torchvision torchaudio --index-url https://download.pytorch.org/whl/cu118
```

Verify GPU is working:

```
python -c "import torch; print(torch.cuda.is_available())" # Must print: True
```

STEP 0C — Install AirSim (Pre-Built Binary)

What are we installing?

AirSim has two parts: (1) the 3D world binary — a pre-compiled executable you just run, and (2) the Python client library — the code that talks to that world from Python. We use the pre-built Blocks binary to avoid installing 50GB of Unreal Engine.

WHY: Pre-built binary saves hours of Unreal Engine setup and works perfectly for development.

Exact Steps:

- Go to: github.com/microsoft/AirSim/releases
- Download: AirSimNH.zip (latest Windows release)
- Extract to C:\AirSim\
- Double-click AirSimNH.exe — a 3D world should open
- Press 1 on keyboard to choose drone mode

```
# Install Python client: pip install airsims # Test connection (with AirSim running): python -c "
import airsims client = airsims.MultirotorClient() client.confirmConnection() print('Connected!')
client.enableApiControl(True) client.armDisarm(True) client.takeoffAsync().join() print('Drone
took off!') "
```

MILESTONE: AirSim is open, Python connects, drone takes off and hovers. Step 0C complete.

STEP 0D — Install PX4 SITL Inside WSL2

Run these commands inside your Ubuntu WSL2 terminal:

```
sudo apt update && sudo apt upgrade -y sudo apt install -y git cmake python3-pip ninja-build git
clone https://github.com/PX4/PX4-Autopilot.git --recursive cd PX4-Autopilot bash
./Tools/setup/ubuntu.sh # Build (takes 5-10 minutes first time): make px4_sitl
gazebo-classic_iris # Success = INFO [px4] Startup script returned successfully
```

NOTE: If build fails, try: sudo apt install -y gcc-arm-none-eabi

STEP 0E — Create Python Virtual Environment

A virtual environment is an isolated folder for one project's packages — prevents conflicts.

```
cd C:\ mkdir hunt_eye && cd hunt_eye python -m venv venv venv\Scripts\activate # Install all
packages: pip install airsims ultralytics opencv-python timm torch torchvision ^
stable-baselines3 gymnasium pymavlink PyQt6 numpy geopy ^ osmnx torchreid rasterio tensorboard
```

MILESTONE: Virtual environment created, all packages installed. Your workshop is ready.

PHASE 1

Project Skeleton — The Foundation

Now we build the empty frame of the house before adding walls and furniture. Every file is a placeholder we fill over the coming phases. The architecture is correct from Day 1.

STEP 1 — Create Folder Structure + config.yaml

```

hunt_eye/ |-- main.py # THE one command that starts everything |-- config.yaml # all settings —
change mode:sim to mode:real here |-- requirements.txt |-- core/ | |-- orchestrator.py # owns
and starts every module | |-- shared_state.py # the shared whiteboard every module reads/writes
| |-- hal.py # Hardware Abstraction Layer — sim or real drone | |-- event_bus.py # pub/sub
messaging | +-- safety.py # checks every command before reaching the drone |-- simulation/ | |--
airsim_client.py # connects to AirSim | +-- terrain_loader.py # DEM + OSM buildings -> cost map
|-- perception/ | |-- camera_stream.py # grabs frames at 30Hz | |-- detector.py # YOLOv8
inference wrapper | +-- depth_estimator.py # MiDaS depth from RGB |-- tracking/ | |-- tracker.py
# ByteTrack wrapper | |-- target_filter.py # selective lock-on logic | +-- reid_engine.py #
OSNet re-identification |-- ai/ | |-- gym_env.py # Gymnasium environment | |-- agent.py #
PPO/SAC policy load + inference | +-- planner.py # A* path planning on cost map |-- control/ |
+-- mavlink_interface.py # velocity + waypoint commands +-- ui/ +-- dashboard.py # PyQt6 live
monitoring window

```

Create with one command (Windows Command Prompt):

```

mkdir core simulation perception tracking ai control ui type nul > main.py type nul >
config.yaml for %d in (core simulation perception tracking ai control ui) do type nul >
%d\__init__.py

```

The config.yaml — Every Setting in One Place

Instead of numbers scattered across 20 files, every setting lives here. Changing from sim to real drone = changing one word: mode: real

```

mode: sim # CHANGE TO 'real' FOR REAL DRONE. That is it. sim: host: "127.0.0.1"
takeoff_altitude: 5.0 camera: width: 640 height: 480 fps: 30 yolo: model: "yolov8n.pt" # n=nano
(fastest on laptop GPU) confidence: 0.45 target_classes: ["person"] # what to look for tracking:
max_lost_frames: 30 reid_threshold: 0.65 terrain: dem_path: "data/elevation.tif" bbox: [72.8,
18.9, 72.95, 19.05] # Mumbai bounding box safety: max_velocity_ms: 5.0 max_altitude_m: 30.0
geofence_radius_m: 100.0 rl: model_path: "models/ppo_hunt_eye.zip" mavlink: connection:
"udp:127.0.0.1:14550"

```

STEP 2 — SharedState + HAL + SystemOrchestrator

shared_state.py — the whiteboard:

```

# core/shared_state.py import threading from dataclasses import dataclass, field import numpy
as np @dataclass class SharedState: lock: threading.RLock =
field(default_factory=threading.RLock) sim_time: float = 0.0 uav_position: np.ndarray =
field(default_factory=lambda: np.zeros(3)) uav_altitude: float = 0.0 uav_gps: tuple = (0.0,
0.0) latest_frame: object = None detections: list = field(default_factory=list) depth_map:
object = None active_target_id: int = -1 target_bbox: tuple = None cost_map: object = None
current_waypoint: object = None planned_path: list = field(default_factory=list) system_mode:
str = "idle"

```

hal.py — the sim/real switch:

```

# core/hal.py class HAL: def __init__(self, config): if config["mode"] == "sim": from
simulation.airsim_client import SimBackend self.backend = SimBackend(config["sim"]) else: from
control.real_backend import RealBackend self.backend = RealBackend(config["real"]) def
get_frame(self): return self.backend.get_frame() def get_pose(self): return
self.backend.get_pose() def send_velocity(self, vx, vy, vz, yaw): return
self.backend.send_velocity(vx, vy, vz, yaw) def send_land(self): return self.backend.send_land()

```

STEP 3 — CameraStream + Dashboard (First Visual Output)

After this step you will see a live camera feed from inside AirSim in a desktop window. This proves the whole pipeline from AirSim to Python to screen is working.

```
# perception/camera_stream.py import threading, time class CameraStream: def __init__(self,
hal, state): self.hal = hal self.state = state def stream_loop(self): while True: frame =
self.hal.get_frame() # from AirSim or webcam with self.state.lock: self.state.latest_frame =
frame time.sleep(1/30) # 30 Hz
```

MILESTONE: python main.py runs, drone hovers in AirSim, PyQt6 window shows live camera feed.

PHASE 2

Vision — Making the Drone See

Now we give the drone eyes — detect objects, estimate distances, and build a map of the terrain.

STEP 4 — YOLOv8 Object Detector

YOLOv8n (nano) is the fastest YOLO model — perfect for an RTX 3050 laptop. ~35fps at 640x480 with GPU. The perception loop calls this at 15Hz.

```
# perception/detector.py from ultralytics import YOLO class Detector: def __init__(self,
config, state): self.model = YOLO(config["model"]) # downloads yolov8n.pt automatically
self.conf = config["confidence"] self.state = state self.target_classes =
config["target_classes"] def detect(self, frame): results = self.model(frame, conf=self.conf,
verbose=False) detections = [] for r in results: for box in r.bboxes: cls_name =
self.model.names[int(box.cls)] if cls_name in self.target_classes: detections.append({ "bbox":
box.xyxy[0].tolist(), "confidence": float(box.conf), "class_name": cls_name }) with
self.state.lock: self.state.detections = detections return detections
```

WHY: We filter by class BEFORE tracking — tracker never even sees irrelevant objects.

STEP 5 — MiDaS Depth Estimation

MiDaS takes the RGB camera frame and produces a depth map — a greyscale image where bright = far, dark = near. Used to estimate how many metres the target is away.

```
# perception/depth_estimator.py import torch, cv2 class DepthEstimator: def __init__(self,
state): self.state = state self.model = torch.hub.load("intel-isl/MiDaS", "MiDaS_small")
self.transform = torch.hub.load("intel-isl/MiDaS", "transforms").small_transform
self.model.eval() if torch.cuda.is_available(): self.model = self.model.cuda() def
estimate(self, frame): img = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB) inp = self.transform(img)
if torch.cuda.is_available(): inp = inp.cuda() with torch.no_grad(): depth = self.model(inp)
depth = torch.nn.functional.interpolate( depth.unsqueeze(1), size=frame.shape[:2],
mode="bicubic", align_corners=False).squeeze() depth_np = depth.cpu().numpy() with
self.state.lock: self.state.depth_map = depth_np return depth_np
```

STEP 6 — Terrain Loader (Cost Map)

Before the sim starts we pre-load terrain data. GDAL reads elevation data. osmnx downloads building footprints from OpenStreetMap. Combined into a 2D cost map: 0 = open air, 100 = building/mountain.

```
# simulation/terrain_loader.py import numpy as np, rasterio, osmnx as ox class TerrainLoader:
def __init__(self, config, state): self.config = config self.state = state def preload(self):
with rasterio.open(self.config["dem_path"]) as dem: elevation = dem.read(1).astype(float) bbox
= self.config["bbox"] buildings = ox.features_from_bbox( bbox[1], bbox[3], bbox[0], bbox[2],
tags={"building": True}) obstacle_map = self._rasterize(buildings, elevation.shape) gradient =
np.gradient(elevation) slope_cost = np.sqrt(gradient[0]**2 + gradient[1]**2) * 10 cost_map =
np.clip(slope_cost + obstacle_map * 100, 0, 100) with self.state.lock: self.state.cost_map =
cost_map print(f"[TerrainLoader] Cost map: {cost_map.shape}")
```

MILESTONE: Dashboard shows: bounding boxes on detected people, depth map, coloured cost map. Phase 2 done.

PHASE 3

Tracking — Locking On to One Target

Detection finds ALL targets. Tracking locks onto ONE and never lets go. This is the 'selective' in Selective Target Tracking.

STEP 7 — ByteTrack Multi-Object Tracker

ByteTrack takes detection boxes from every frame and figures out which box in frame 50 is the same person as in frame 49. Uses Kalman filter + IoU matching. Assigns a permanent integer ID to each person.

```
# Install ByteTrack first: # git clone https://github.com/ifzhang/ByteTrack.git # cd ByteTrack
&& pip install -r requirements.txt && pip install -e . # tracking/tracker.py from
bytetrack.byte_tracker import BYTETTracker import numpy as np class Tracker: def __init__(self,
config, state): self.state = state self.tracker = BYTETTracker( track_thresh=0.5, # min
confidence to track track_buffer=30, # frames to keep a lost track alive match_thresh=0.8 # IoU
threshold for matching ) def update(self, detections, frame_id): if not detections: return []
dets = np.array([[*d["bbox"], d["confidence"]] for d in detections]) tracks =
self.tracker.update(dets, frame_id=frame_id) with self.state.lock: self.state.all_tracks =
tracks return tracks
```

STEP 8 — TargetFilter (The Selective Part)

ByteTrack gives ALL tracks. TargetFilter picks ONE and commits to it. On first detection, picks highest-confidence track. Then verifies every frame the target still matches. If lost, triggers re-ID.

```
# tracking/target_filter.py class TargetFilter: def __init__(self, state): self.state = state
def select(self, tracks, frame): with self.state.lock: active_id = self.state.active_target_id
if active_id == -1: if tracks: best = max(tracks, key=lambda t: t.score) with self.state.lock:
self.state.active_target_id = best.track_id self.state.target_bbox = best.tlwh print(f"Locked
onto track ID {best.track_id}") else: matched = [t for t in tracks if t.track_id == active_id]
if matched: with self.state.lock: self.state.target_bbox = matched[0].tlwh else: with
self.state.lock: self.state.system_mode = "reacquiring" print("Target lost — starting
re-identification")
```

MILESTONE: One target tracked with persistent ID that does not change when others enter frame. Phase 3 done.

PHASE 4

Control — Making the Drone Actually Move

The drone stops being a spectator and becomes a hunter. We implement a PID controller first — a classic algorithm that keeps the target centred in frame by adjusting drone velocity.

STEP 9 — MAVLink Interface + PID Controller

What is PID?

PID = Proportional-Integral-Derivative. A formula from the 1940s still used in everything from thermostats to rockets. If the target is left of frame centre, fly left. The further left the target, the faster we fly (Proportional). If we keep slightly missing, gradually increase correction (Integral). If correcting too fast and about to overshoot, slow down (Derivative).

```
# control/mavlink_interface.py import pymavlink.mavutil as mavutil, time
class MavlinkInterface:
    def __init__(self, config, state):
        self.conn = mavutil.mavlink_connection(config["connection"])
        self.conn.wait_heartbeat()
        self.state = state
        self.Kp = 0.002 # proportional - main correction strength
        self.Ki = 0.0001 # integral - drift correction
        self.Kd = 0.001 # derivative - overshoot damping
        self._integral = [0, 0]
        self._prev_error = [0, 0]
    def control_loop(self):
        while True:
            with self.state.lock:
                bbox = self.state.target_bbox
                mode = self.state.system_mode
                frame = self.state.latest_frame
                if bbox is None or mode != "tracking" or frame is None:
                    time.sleep(1/20)
                    continue
                frame_cx = frame.shape[1] / 2
                frame_cy = frame.shape[0] / 2
                target_cx = bbox[0] + bbox[2] / 2
                target_cy = bbox[1] + bbox[3] / 2
                error_x = target_cx - frame_cx # + = target right of centre
                error_y = target_cy - frame_cy # + = target below centre
                self._integral[0] += error_x
                self._integral[1] += error_y
                vx = self.Kp*error_x + self.Ki*self._integral[0] + self.Kd*(error_x-self._prev_error[0])
                vy = self.Kp*error_y + self.Ki*self._integral[1] + self.Kd*(error_y-self._prev_error[1])
                vx = max(-2.0, min(2.0, vx)) # cap speed for safety
                vy = max(-2.0, min(2.0, vy))
                self._send_velocity(vx, vy, -0.1, 0) # -0.1 vz = slowly descend
            self._prev_error = [error_x, error_y]
            time.sleep(1/20)
```

MILESTONE: Drone moves in AirSim to keep target centred. Target walks left — drone follows left. Phase 4 done.

PHASE 5

AI Brain — Replacing Rules With Intelligence

PID works but is dumb — it only reacts, never plans. If the target runs around a building, PID flies into the wall. Reinforcement Learning trains an AI to anticipate, plan, and optimise.

STEP 10 — Gymnasium Environment Wrapper

Before training an RL agent, we wrap our simulation as a Gymnasium environment: what the AI sees (observations), what it can do (actions), how it scores (rewards).

```
# ai/gym_env.py import gymnasium as gym import numpy as np class HuntEyeEnv(gym.Env): def
__init__(self, orchestrator): self.orch = orchestrator self.state = orchestrator.state #
OBSERVATIONS: 8 numbers the AI sees each step self.observation_space = gym.spaces.Box(low=-1,
high=1, shape=(8,), dtype=np.float32) # ACTIONS: [vx, vy, vz, yaw_rate] all in range [-1, 1],
scaled to 5 m/s self.action_space = gym.spaces.Box(low=-1, high=1, shape=(4,),
dtype=np.float32) def step(self, action): vx, vy, vz, yaw = action * 5.0
self.orch.hal.send_velocity(vx, vy, vz, yaw) import time; time.sleep(0.05) reward = 0 with
self.state.lock: bbox = self.state.target_bbox altitude = self.state.uav_altitude if bbox is
not None: cx_err = abs((bbox[0] + bbox[2])/2) / 640 - 0.5 cy_err = abs((bbox[1] + bbox[3])/2) /
480 - 0.5 reward += 1.0 - (cx_err + cy_err) # centred = +1.0 box_area = bbox[2] * bbox[3]
reward += min(box_area / (640*480), 1.0) * 0.5 # close = +0.5 else: reward -= 2.0 # heavy
penalty for losing target if altitude < 1.0: return self._obs(), reward, True, False, {} #
crashed return self._obs(), reward, False, False, {}
```

STEP 11 — PPO Training

PPO (Proximal Policy Optimization) is the training algorithm. We run the simulation for millions of steps. Each step the AI tries an action, gets a reward, and updates its policy to do better next time.

```
# train_agent.py – run this separately (leave overnight) from stable_baselines3 import PPO from
ai.gym_env import HuntEyeEnv from core.orchestrator import SystemOrchestrator import yaml
config = yaml.safe_load(open("config.yaml")) orch = SystemOrchestrator(config) env =
HuntEyeEnv(orch) model = PPO("MlpPolicy", env, verbose=1, learning_rate=3e-4, n_steps=2048,
batch_size=64, gamma=0.99, tensorboard_log="./logs/") model.learn(total_timesteps=1_000_000)
model.save("models/ppo_hunt_eye") print("Training complete!") # Monitor in real-time:
tensorboard --logdir ./logs/
```

NOTE: Training 1M steps takes 2-4 hours on RTX 3050. Leave AirSim running. Check TensorBoard for progress.

STEP 12 — A* Path Planner on Cost Map

A* (A-star) is a path planning algorithm from 1968 still considered state of the art. Finds the shortest path from A to B on the cost map, avoiding high-cost cells (buildings, terrain). RL agent says 'go to this coordinate' — A* figures out the safest route.

```
# ai/planner.py import heapq, numpy as np class Planner: def __init__(self, state): self.state =
state def plan(self, start, goal): with self.state.lock: cost_map = self.state.cost_map if
cost_map is None: return [goal] open_set = [(0, start)] came_from = {} g_score = {start: 0}
while open_set: _, current = heapq.heappop(open_set) if current == goal: path = [] while current
in came_from: path.append(current); current = came_from[current] path.reverse() with
self.state.lock: self.state.planned_path = path return path for dx,dy in
[(0,1),(0,-1),(1,0),(-1,0),(1,1),(1,-1),(-1,1),(-1,-1)]: nb = (current[0]+dx, current[1]+dy) if
not (0<=nb[0]<cost_map.shape[0] and 0<=nb[1]<cost_map.shape[1]): continue if cost_map[nb] >=
90: continue # blocked cell step = np.sqrt(dx**2+dy**2) * (1 + cost_map[nb]/10) new_g =
g_score[current] + step if nb not in g_score or new_g < g_score[nb]: g_score[nb] = new_g h =
np.sqrt((goal[0]-nb[0])**2 + (goal[1]-nb[1])**2) heapq.heappush(open_set, (new_g+h, nb))
came_from[nb] = current return []
```


MILESTONE: RL policy replaces PID. Drone navigates around obstacles. A* path shown on dashboard.
Phase 5 done.

PHASE 6

Re-ID, Safety Layer & Real Drone Deployment

Final phase: re-acquisition when target hides, safety checks before every command, and the sim-to-real switch. After this, Hunt Eye is complete.

STEP 13 — OSNet Re-Identification Engine

When the target goes behind a building and reappears, ByteTrack gives them a new ID. OSNet solves this: on first lock it creates a 512-number appearance fingerprint. On re-acquisition, compares fingerprints with cosine similarity. If similarity > 0.65, that is our target — re-lock and continue.

```
# tracking/reid_engine.py import torchreid, torch, numpy as np, cv2 class ReIDEngine: def
__init__(self, config, state): self.state = state self.threshold = config["reid_threshold"]
self.target_embedding = None self.model = torchreid.models.build_model( name="osnet_x1_0",
num_classes=1000, pretrained=True) self.model.eval() if torch.cuda.is_available(): self.model =
self.model.cuda() def store_embedding(self, frame, bbox): crop = self._crop(frame, bbox)
self.target_embedding = self._embed(crop) def find_target(self, frame, detections): if
self.target_embedding is None or not detections: return None best_match, best_sim = None, 0 for
det in detections: emb = self._embed(self._crop(frame, det["bbox"])) sim =
float(np.dot(self.target_embedding, emb) /
(np.linalg.norm(self.target_embedding)*np.linalg.norm(emb)+1e-8)) if sim > best_sim: best_sim,
best_match = sim, det if best_sim >= self.threshold: print(f"Target re-acquired! Similarity:
{best_sim:.3f}") return best_match return None
```

STEP 14 — Safety Layer (Non-Negotiable)

The safety layer is the LAST thing checked before EVERY command reaches the drone. No command bypasses it. In simulation, a bug restarts the program. On a real drone, a bug means it crashes into a person. These 5 checks prevent that:

- **Heartbeat check** — if laptop loses connection for over 2 seconds, drone lands immediately
- **Velocity cap** — drone cannot exceed 5 m/s regardless of what AI commands
- **Altitude cap** — drone cannot go above 30 metres
- **Geofence** — drone cannot leave a 100m radius from launch point
- **Emergency land** — any module triggers it by writing `system_mode = 'emergency'`

```
# core/safety.py import time, geopy.distance class SafetyLayer: def __init__(self, config):
self.max_vel = config["safety"]["max_velocity_ms"] self.max_alt =
config["safety"]["max_altitude_m"] self.geo_r = config["safety"]["geofence_radius_m"]
self.home_gps = None self.last_hb = time.time() def validate_and_send(self, command, hal,
state): if time.time() - self.last_hb > 2.0: hal.send_land(); return with state.lock: if
state.system_mode == "emergency": hal.send_land(); return command.vx = max(-self.max_vel,
min(self.max_vel, command.vx)) command.vy = max(-self.max_vel, min(self.max_vel, command.vy))
command.vz = max(-self.max_vel, min(self.max_vel, command.vz)) with state.lock: if
state.uav_altitude > self.max_alt: command.vz = 0.5 gps = state.uav_gps if self.home_gps and
gps != (0,0): if geopy.distance.distance(self.home_gps, gps).meters > self.geo_r:
hal.send_return_to_launch(); return hal.send_velocity(command.vx, command.vy, command.vz,
command.yaw)
```

STEP 15 — Sim-to-Real Switch

Everything worked in simulation. Now we flip the switch. Because the HAL hides all differences between sim and real, the switch is literally one word in `config.yaml`:

```
# config.yaml mode: real # was: sim real: serial_port: "/dev/ttyUSB0" # Linux — or COM3 on
Windows baud_rate: 57600 # Then just run: python main.py --config config.yaml # System opens
serial port, reads real camera, arms real drone. Same code.
```

Component	Transfers?	Notes
YOLOv8	YES	Pixels are pixels — may need fine-tuning on real lighting
ByteTrack	YES	Pure math, works on any video stream
OSNet Re-ID	YES	Appearance fingerprinting is camera-agnostic
A* Planner	YES	Same cost map
Safety layer	YES	More important in real mode
RL policy	MOSTLY	Domain randomisation during training handles physics gap
Camera calibration	ONE-TIME	Each real camera needs one lens calibration
MAVLink	CHANGED	Serial port instead of UDP — handled by RealBackend

MILESTONE: FINAL: python main.py — drone takes off, finds target, locks on with persistent ID, follows around obstacles, re-acquires after occlusion, all with safety checks. Change config to real — same code works on a physical drone.

APPENDIX — QUICK REFERENCE

All Install Commands (single block):

```
pip install airsim ultralytics opencv-python timm torch torchvision torchaudio ^
stable-baselines3 gymnasium pymavlink PyQt6 numpy geopy ^ osmnx torchreid rasterio tensorboard
# ByteTrack (not on pip): git clone https://github.com/ifzhang/ByteTrack.git cd ByteTrack &&
pip install -r requirements.txt && pip install -e . cd .. # PX4 SITL (inside WSL2): git clone
https://github.com/PX4/PX4-Autopilot.git --recursive cd PX4-Autopilot && bash
./Tools/setup/ubuntu.sh make px4_sitl gazebo-classic_iris
```

Run Commands:

```
# 1. Always start AirSim first: C:\AirSim\AirSimNH.exe # 2. Activate environment + run Hunt Eye:
venv\Scripts\activate python main.py --config config.yaml # 3. Train RL agent (separate
session): python train_agent.py # 4. Monitor training: tensorboard --logdir ./logs/ # 5. Switch
to real drone – edit config.yaml: # mode: real # Then run: python main.py --config config.yaml
```

Glossary:

AirSim	3D simulation tool by Microsoft. Like a realistic video game your code controls.
CUDA	Nvidia software that lets Python use the GPU for fast math.
DEM	Digital Elevation Model — file containing terrain height at each point.
Geofence	Invisible GPS boundary the drone cannot cross.
Gymnasium	Standard way to define RL training environments, made by OpenAI.
HAL	Hardware Abstraction Layer — hides whether we use sim or real drone.
IoU	Intersection over Union — measures how much two bounding boxes overlap (0-1).
MAVLink	Micro Air Vehicle Link — communication protocol drones understand.
OSNet	Neural network that identifies people by appearance (clothes, body shape).
PID	Proportional-Integral-Derivative — classic formula that keeps systems stable.
PPO	Proximal Policy Optimization — reliable reinforcement learning algorithm.
Re-ID	Re-Identification — recognising a person after they left the camera view.
RL	Reinforcement Learning — training AI through trial, reward, and repetition.
SITL	Software In The Loop — flight controller firmware running on a computer.
YOLO	You Only Look Once — object detection AI that processes frames at 30+ fps.

Hunt Eye — 100% open source. 100% free. Your job is to connect the pieces.